Wprowadzenie oraz obsługa systemu plików

Początek pracy, logowanie do systemu

Podczas tego ćwiczenia poznamy podstawy pracy z systemem Linux przy wykorzystaniu najpopularniejszej, domyślnej powłoki *bash*. Aby uzyskać dostęp do powłoki, należy uruchomić aplikację *Terminal* z menu startowego interfejsu graficznego systemu.

Po uruchomieniu terminala wyświetlana jest linia poleceń wraz z nazwą użytkownika (*put*), nazwą maszyny (*put*) oraz znak zachęty $. Oznacza to, że zalogowany użytkownik jest użytkownikiem zwykłym. Oprócz tego wyróżniamy, użytkownika uprzywilejowanego *root* posiadającego pełne prawa administracyjne do systemu. W takim wypadku, znak zachęty zmienia się na #, a nazwa użytkownika na *root*. Po znaku zachęty możemy wprowadzać kolejne polecenia zatwierdzanie klawiszem *Enter*.

Wydawanie poleceń

Polecenia w systemie Linux mogą przyjmować **argumenty** oraz **przełączniki**, zgodnie ze wzorcem:

```
polecenie [przełączniki] [argumenty]
```

Oto przykład:

```
ls -al /etc
```

W tym przypadku polecenie `ls` zostało uruchomione z argumentem `/etc` oraz z przełącznikami `al`. Wydanie polecenia `ls` bez argumentu spowoduje wyświetlenie zawartości aktualnego katalogu; podanie natomiast jako argumentu nazwy katalogu, spowoduje wyświetlenie zawartości tego, wskazanego katalogu - w przykładzie jest to katalog `/etc`. Dalej, domyślne działanie polecenia można modyfikować stosując przełączniki, które są przeważnie jednoznakowymi skrótami, zawsze występującymi po znaku "`-`". W przykładzie zastosowano dwa przełączniki `a` i `l`. Opis możliwych do zastosowania argumentów i przełączników można odnaleźć w pomocy systemowej dla właściwego polecenia. Przełączniki, jak widać, można łączyć - kolejny przykład przedstawia jak można wydawać polecenie z przełącznikami:

```
ls -a -l /etc
```

```
ls -l -a /etc
```

```
ls -al /etc
```

```
ls -la /etc
```

Każde z pokazanych powyżej poleceń jest równoważne i skutkuje wykonaniem tej samej operacji. Przełączniki wieloznakowe podawane są z wykorzystaniem dwuznaku `--`.

Garść porad i wskazówek:

Dla efektywnej pracy z terminalem warto poznać kilka przydatnych "sztuczek":

- Kopiowanie zaznaczonego tekstu z terminala `Ctrl + Shift + C`
- Wklejanie tekstu do terminala `Ctrl + Shift + V`
- Przerywanie działania aktualnie wykonywanego polecenia `Ctrl + C`
- Podczas wpisywania poleceń, ścieżek dostępu itp. naciskając klawisz `Tab` terminal uzupełni dalszą część polecenia, jeśli jest tylko jedno możliwe dopełnienie. Jeśli możliwości jest kilka, dwukrotne naciśnięcie `Tab` wyświetli możliwe dopełnienia.
- Naciskając klawisze strzałek góra-dół w trakcie pracy z linią poleceń, możemy w łatwy sposób poruszać się pomiędzy wydawanymi w tej sesji poleceniami.
- Szerszy dostęp do historii komend uzyskamy naciskając `Ctrl+R` i wyszukując odpowiednie polecenie
- Polecenie `clear` powoduje wyczyszczenie ekranu terminala.
- Wpisanie `!!` automatycznie zastępuje ostatnio wydane polecenie. Przykładowo, jeżeli zapomnimy wykonać skomplikowanej komendy z uprawnieniami użytkownika *root*, wystarczy, że w kolejnym kroku wpiszemy `sudo !!`.
- Aby wydać kilka poleceń w jednej linii, możemy odseparować kolejne komendy znakami `&&` lub `;`. W pierwszym przypadku, kolejne polecenia będą wykonywać się tylko wtedy, gdy poprzednie nie zakończą się błędem. Oddzielenie średnikiem powoduje sekwencyjne wykonanie komend bez zwracania uwagi na rezultat operacji.

Pomoc systemowa

W systemie Linux dla każdego polecenia dostępna jest pomoc systemowa opisująca różne aspekty i zastosowania danego narzędzia oraz dostępne przełączniki. Jednym z mechanizmów pozwalających na wyświetlenie plików pomocy jest wywołanie polecenia z przełącznikiem `--help`, np:

```
ls --help
```

powoduje wyświetlenie opisu działania wszystkich przełączników, odniesień do dokumentcji zewnętrznej i innych przydatnych informacji.

Bardziej rozbudowaną dokumentację poleceń można uzyskać poprzez wywołanie:

```
man nazwa_polecenia
```

Powoduje to uruchomenie interaktywnej przeglądarki plików pomocy dla podanego programu lub usługi. Pomoc systemowa wyświetlana jest za pomocą

przeglądarki *more*, którą obsługuje się za pomocą następujących poleceń klawiszowych:

- `Spacja` - przejście do następnej strony;
- `Ctrl+B` - przejście do poprzedniej strony;
- `q` - zamknięcie i opuszczenie przeglądarki;
- `/` - wyszukiwanie tekstu w przód, po znaku / należy wpisać tekst do wyszukania;
- `?` - wyszukiwanie w tył;
- `n`, `N` - przejście do następnego (n) lub poprzedniego (N) wystąpienia poszukiwanego wyrażenia.

Zadanie 1: Nawigacja po systemie Zapoznaj się tutorialem oraz wykonaj przykłady z zakładki `Activities` znajdującej się na końcu strony:

Tutorial nawigacja

Struktura katalogów

Katalog jest strukturą, która umożliwia porządkowanie i grupowanie danych, jakie są przechowywane na dysku komputera. Katalogi mają strukturę hierarchiczną - w każdym katalogu mogą być przechowywane zarówno pliki, jaki i katalogi niższych poziomów. W systemach UNIX wszystkie dostępne systemy plików postrzegane są przez użytkownika jako jedna struktura katalogów, której wierzchołkiem jest tzw. katalog główny, oznaczany symbolem " `/` ". Jeśli w systemie dostępnych jest kilka systemów plików to jeden z nich jest systemem głównym, a pozostałe są do niego montowane. Regułą jest także, że każdy użytkownik posiada tzw. katalog domowy, zwykle jest to podkatalog katalogu `home`, w którym użytkownik może przechowywać własne pliki.

Postać systemu plików systemów UNIX jest w dużej części ustandaryzowana - oto najistotniejsze podkatalogi katalogu głównego:

- `/bin` - katalog zawierający programy niezbędne do uruchomienia systemu;
- `/dev` - katalog zawierający pliki specjalne, które reprezentują dostępne urządzenia;
- `/etc` - katalog z lokalnymi plikami konfiguracyjnymi systemu;
- `/home` - w tym katalog znajdują się podkatalogi domowe użytkowników systemu;
- `/proc` - wirtualny system plików, który dostarcza informacji o bieżących procesach w systemie i jego jądrze;
- `/root` - zwyczajowo katalog domowy użytkownika *//root//*, czyli administratora systemu;
- `/usr` - katalog zawierający zestaw oprogramowania użytkowego dostępnego dla użytkowników;

- `/var` - katalog ten zawiera pliki, które często zmieniają swoją zawartość i/lub rozmiar.

Informacja o katalogu bieżącym jest wyświetlana po wydaniu polecenia:

`pwd`

Jak wspomniano symbol `/` oznacza katalog główny, używa się także innych symboli dla określenia wybranych katalogów:

- `.` - oznacza katalog bieżący,
- `..` - oznacza katalog bezpośrednio nadrzędny;
- `~` - oznacza katalog domowy użytkownika.

Obsługa katalogów

Podstawowe operacje obsługi katalogów można realizować z wykorzystaniem następujących poleceń:

- `cd [przełączniki] nazwa_katalogu` - zmiana katalogu, np.:
  - `cd /etc` - zmienia katalog bieżący na katalog `/etc`;
  - `cd ~` - zmienia katalog bieżący na katalog domowy użytkownika;
  - `cd ..` - zmienia katalog bieżący na katalog bezpośrednio nadrzędny.
- `ls [przełączniki] [nazwa_katalogu]` - wyświetlenie zawartości katalogu, np.:
  - `ls` - wyświetla zawartość katalogu bieżącego;
  - `ls -a` - wyświetla zawartość katalogu bieżącego uwzględniając wszystkie pliki - tzn. także te, których nazwa zaczyna się od znaku `.` (umownie są to pliki ukryte);
  - `ls -l` - wyświetla wszystkie pliki z katalogu bieżącego z uwzględnieniem tzw. "długiego formatu", czyli podając typ każdego obiektu w katalogu (pierwszy znak linii: `d` - katalog, znak `-` - plik zwykły, `l` - dowiązanie), prawa dostępu, liczbę dowiązań, właściciela, nazwę grupy, rozmiar (w bajtach), data ostatniej modyfikacji oraz nazwę
  - `ls -al ~` - jak wyżej, przy czym wyświetlana jest zawartość katalogu domowego wraz z plikami ukrytymi;
  - `ls -al /etc` - jak wyżej, ale wyświetlana jest zawartość katalogu `/etc`.
- `mkdir [przełączniki] nazwa_katalogu` - tworzenie katalogów, np.:
  - `mkdir ./xyz` - utworzenie katalogu `xyz` w katalogu bieżącym;
  - `mkdir ../xyz` - utworzenie katalogu `xyz` w katalogu bezpośrednio nadrzędnym.
- `rmdir [przełączniki] nazwa_katalogu` - usuwanie katalogów, np.:
  - `rmdir ~/xyz` - usunięcie katalogu `xyz` z katalogu domowego;
  - `rmdir ./xyz` - usunięcie katalogu `xyz` z katalogu bieżącego.

Obsługa plików

Plik to zdefiniowana (przeważnie przez użytkownika) porcja danych, która jest przechowywana w systemie w pamięci masowej. W systemach UNIX niemal wszystko jest plikiem, także urządzenia są reprezentowane przez specjalne pliki. Pozwala to, na zachowanie spójnego sposobu dostępu i obsługi do wielu heterogenicznych zasobów, w jeden transparentny sposób. Nazwy plików nie mają podziału na nazwę i rozszerzenie, jednakże można takie podejście stosować; możliwe jest stosowanie w nazwach plików znaków specjalnych (np.: `$`, `%` lub `#`), ale nie jest to zalecane.

Podstawowe operacje obsługi plików można realizować z wykorzystaniem następujących poleceń:

- `cp [przełączniki] nazwa_pliku nowa_nazwa_lub_katalog` - kopiowanie pliku określonego przez pierwszy argument pod nazwę lub do katalogu określonego drugim argumentem, np.:
  - `cp abc.txt xyz.txt` - kopiuje plik `abc.txt` pod nową nazwę `xyz.txt` w katalogu bieżącym;
  - `cp /tmp/abc.txt ~` - kopiuje plik `abc.txt` z katalogu `/tmp` do katalogu domowego użytkownika;
  - `cp abc.txt ~/xyz.txt` - kopiuje plik `abc.txt` z katalogu bieżącego pod nową nazwę `xyz.txt` w katalogu domowym użytkownika.

Przydatnym przełącznikiem polecenia `cp` jest przełącznik `-r`, który służy do kopiowania całych struktur katalogów.

- `rm [przełączniki] lista_plików` - usuwanie plików podanych jako argumenty wywołania, np.:
  - `rm abc.txt xyz.txt` - usuwa pliki `abc.txt` i `xyz.txt` w katalogu bieżącym
  - `rm /tmp/abc.txt` - usuwa plik `abc.txt` z katalogu `/tmp`

Przydatnym przełącznikiem polecenia `rm` jest przełącznik -r, który służy do usuwania całych struktur katalogów.

- `mv [przełączniki] nazwa_pliku nowa_nazwa` - zmiana nazwy pliku określonego pierwszym argumentem wywołania na nazwę określoną drugim argumentem wywołania. Jeśli drugi argument wywołania jest katalogiem, to wówczas plik zostanie przeniesiony do tego katalogu, np.:
  - `mv abc.txt xyz.txt` - zmiana nazwy pliku `abc.txt` na nazwę `xyz.txt` w katalogu bieżącym;
  - `mv /tmp/abc.txt ~` - przeniesienie pliku `abc.txt` z katalogu `/tmp` do katalogu domowego użytkownika.
- `touch [przełączniki] nazwa_pliku` - modyfikuje informacje na temat czasów modyfikacji i odczytu pliku, ale pozwala także na utworzenie pliku, np.:

– `touch abc.txt` - utworzenie (pustego) pliku `abc.txt` w katalogu bieżącym.

Polecenia dotyczące plików (i katalogów) można także wydawać z wykorzystaniem tzw. wzorców uogólniających, które tworzy się z zastosowaniem następujących operatorów:

`*` - zastępuje dowolny ciąg znaków (także pusty),

`?` - zastępuje dokładnie jeden dowolny znak,

`[<znaki>]` - zastępuje dokładnie jeden znak z podanego zakresu, np.: `[xyz]`

`[^<znaki>]` - znak ^ na początku oznacza dopełnienie zbioru, czyli dla przykładu `[^xyz]`, oznacza dowolny znak nie będący literą `x, y` i `z`.

Oto przykładowe polecenia z wykorzystaniem wzorców uogólniających:

`cp ./*.txt ~` - kopiowanie wszystkich plików z rozszerzeniem .txt z katalogu bieżącego do katalogu domowego użytkownika,

`rm ~/[0-9]*` - usunięcie wszystkich plików z katalogu domowego, których nazwa rozpoczyna się od cyfry.

Zadanie 2: Manipulacja plikami Zapoznaj się tutorialem oraz wykonaj polecenia z zakładki Activities znajdującej się na końcu strony. Po skończonym ćwiczeniu usuń pliki i foldery które stworzyłeś:

Tutorial

Instalacja pakietów i inne narzędzia

W dystrybucjach systemu Linux oprogramowanie instaluje się z centralnych repozytoriów. Należy unikać "zaśmiecania" systemu za pomocą zewnętrznych instalatorów pobieranych ze stron internetowych (na przykład w postaci plików *.run*).

W dystrybucji Ubuntu do instalowania paczek służy narzędzie `apt`. Część programów (zazwyczaj okienkowych) jest również dostępna jako pakiety `snap`. Ubuntu posiada graficzny "sklep" z aplikacjami o nazwie *Software*.

Zadanie 3: Inne narzędzia

1. Uzyj komendy `history` i sprawdź za co jest odpowiedzialna.
2. Uzyj komendy `top` żeby zobaczyć listę procesów w systemie i jej alternatywę `htop`.
3. Jeżeli htop jest niedostępny, użyj `sudo apt-get install htop`
4. Wykonaj polecenie `apt --help`. Zapoznaj się z dostępnymi opcjami. Spróbuj wyszukać dowolny pakiet (na przykład *neovim*).

Odczyt i edycja tekstu: `cat`, `nano`, `vim`

W pracy z systemami, które nie posiadają środowiska graficznego lub poprzez zdalny terminal często zachodzi konieczność edycji plików tekstowych z poziomu terminala. Odczyt pliku mozna wykonać za pomocą komendy:

```
cat plik.txt
```

Sprawna edycja plików tekstowych możliwa jest dzięki konsolowym edytorom takim jak *Emacs*, *Vim* czy *Nano*.

Osoby posługujące się takimi edytorami na co dzień znają mnóstwo skrótów i trików, które powodują, że praca z takim programem może być sprawniejsza niż z edytorem graficznym.

Nano

Na potrzeby sporadycznej edycji plików, np. konfiguracyjnych, najprostszy w użyciu jest program *Nano*. Oferuje on quasi-graficzny interfejs z podpowiedziami skrótów klawiszowych.

Przykładowe użycie (jeśli plik nie istnieje, zostanie utworzony):

```
nano plik.txt
```

Najważniejsze skróty:

- **Ctrl-o** - zapisz
- **Ctrl-x** - wyjdź
- **Ctrl-k** - wytnij tekst (domyślnie bieżąca linia)
- **Ctrl-u** - wklej tekst

Vim/Neovim

Bardziej sprawna (aczkolwiek wymagająca pewnej wstępnej wiedzy) edycja plików jest możliwa w edytorze *Vim*. Istnieje też jego alternatywna wersja: *Neovim*.

Przykładowe użycie (jeśli plik nie istnieje, zostanie utworzony):

```
vim plik.txt
```

lub dla wersji *Neovim*:

```
nvim plik.txt
```

Najważniejsze skróty klawiszowe:

- `i` - przechodzi w tryb edycji, pozwala na wprowadzenie tekstu
- `a` - przechodzi w tryb edycji za kursorem, pozwala na wprowadzanie tekstu
- `Esc` - wychodzi z trybu edycji lub przerywa wpisywanie komendy

Komendy (zatwierdzane enterem):

- `:w` - zapisz
- `:q` - wyjdź
- `:q!` - wyjdź bez zapisywania
- `:x` - zapisz i wyjdź

Zadanie 4: Edytor tekstu

1. Korzystając z *Vim*-a stwórz i wyedytuj dowolny plik tekstowy.
2. Wyświetl plik tekstowy za pomocą komendy `cat`.
3. Korzystając z *nano* wyedytuj plik tekstowy.
4. Usuń plik który stworzyłeś

Zdalny dostęp do urządzeń

System Linux jest wyposażony w narzędzia umożliwiające zdalny dostęp do konsoli innego urządzenia oraz transfer plików.

Polecenie `ssh`

Polecenie `ssh` służy do uzyskiwania zdalnego dostępu do innego urządzenia. Składnia polecenia wygląda następująco:

```
ssh <user>@<address>
```

na przykład:

```
ssh put@127.0.0.1
```

podłączy nas do naszego własnego komputera.

Jeśli nazwa użytkownika na zdalnym urządzeniu jest identyczna z naszą nazwą użytkownika możemy ją pominąć w komendzie:

```
ssh 127.0.0.1
```

Adres urządzenia można sprawdzić przy pomocy komendy `ip`.

Aby wyjść z sesji na zdalnym komputerze, należy wydać polecenie `exit` lub nacisnąć kombinację klawiszy `Ctrl + D`.

Transfer plików pomiędzy urządzeniami

Podstawowym poleceniem służącym do transferu plików między urządzeniami jest `scp`. Składnia jest połączeniem cp oraz ssh:

```
scp <local_file> <user>@<address>:<target_directory>
scp -r <local_directory> <user>@<address>:<target_directory>

scp <user>@<address>:<remote_file> <local_directory>
scp -r <user>@<address>:<remote_directory> <local_directory>
```

Zadanie 5: Zdalny dostęp

1. Utwórz w swoim katalogu plik tekstowy o nazwie składającej się z Twojego imienia i nazwiska. Poproś prowadzącego o adres IP serwera, a następnie skopiuj utworzony plik za pomocą komendy `scp` do katalogu `~/ssh_test`. Następnie zaloguj się do serwera zdalnie i zmień zawartość skopiowanego pliku. Skopiuj zmieniony plik z powrotem na swój komputer z poziomu swojego komputera (a nie serwera).

Filtry i potoki

Filtry

Istnieją programy, których zadaniem jest odczyt danych ze standardowego wejścia, przetworzenie tych danych i ich zapis na standardowe wyjście. Programy takie nazywane są filtrami i są szeroko wykorzystywane w przetwarzaniu potokowym. Poniżej przedstawiono najczęściej wykorzystywane filtry:

- `cat` - najprostszy filtr, nie wprowadzający zmian do przetwarzanych danych. Użyteczne przełączniki:
  - `-s` z paru pustych linii robi jedną
  - `-n` numeruje wszystkie linie
  - `-b` numeruje niepuste linie
  - `-A` pokazuje znaki specjalne
- `head` - wyświetla początkową część pliku o podanej nazwie lub danych wejściowych otrzymanych z potoku gdy nazwa pliku nie jest podana. Standardowo wyświetlanych jest pierwszych 10 linii odczytanych danych. Używając przełączników liczbę tą można zmienić:
  - `-c` pozwala określić liczbę wyświetlanych znaków
  - `-n` pozwala określić liczbę wyświetlanych linii
- `tail` - wyświetla końcową część pliku o podanej nazwie lub danych wejściowych otrzymanych z potoku, gdy nazwa pliku nie jest podana,. Standardowo wyświetlanych jest ostatnich 10 linii danych. Używając przełączników liczbę tą można zmienić:
  - `-c` pozwala określić liczbę wyświetlanych znaków
  - `-n` pozwala określić liczbę wyświetlanych linii
  - `-f` powoduje, że dane po zapisaniu ich do pliku są wyświetlane na standardowym wyjściu

- `sort` - służy do sortowania danych wejściowych, które to domyślnie sortowane są leksykograficznie. Sortowanie danych odbywa się liniami. Najważniejsze przełączniki:
  - `-n` pozwala sortować numerycznie
  - `-b` ignoruje przy sortowaniu spacje znajdujące się na początku linii
  - `-t` pozwala zmienić domyślny separator kolumn, którym są znaki tabulacji lub spacji
  - `-f` pozwala ignorować przy sortowaniu wielkość liter
  - `-r` odwraca kolejność sortowania
  - `+liczba` pozwala by sortowanie odbywało się względem dowolnej kolumny (a nie początku linii). Liczba określa liczbę kolumn pominiętych przy sortowaniu
  - `-o nazwa_pliku` zapisuje rezultat sortowania do pliku o podanej nazwie
- `grep [opcje] wyrażenie [lista_plików]` - przeszukuje dane pochodzące ze standardowego wejścia lub pliki wyszczególnione na liście plików, wypisując tylko linie zawierające szukane wyrażenie. Szukane wyrażenie zapisywane jest za pomocą wyrażenia regularnego. Najważniejsze przełączniki:
  - `-v` wyszukuje linie nie zawierające szukanego wzorca
  - `-c` podaje liczbę odszukanych wyrażeń
  - `-i` ignoruje wielkość liter przy wyszukiwaniu
  - `-n` wyświetla numery linii zawierających dany wzorzec
  - `-h` przy wyświetlaniu linii zawierających szukany wzorzec pomija nazwy plików
  - `-r` pozwala na przeszukiwanie rekurencyjne, np. `grep wzorzec -r katalog`
  - `-l` pokazuje nazwy plików zawierających określony wzorzec
  - `-L` pokazuje nazwy plików nie zawierających określonego wzorca

Zasady konstrukcji podstawowych wyrażeń regularnych opisujących szukany wzorzec są następujące:

- `.` - reprezentuje dowolny znak
- `[abc]` - oznacza jeden ze znaków a, b lub c
- `[a-z]` - oznacza jeden ze znaków z podanego zakresu
- `[^0-9]` - oznacza dopełnienie podanego zbioru (znaki inne niż od `0` do `9`)
- `*` - reprezentuje powtórzenie dowolną liczbę razy (0 lub więcej) wyrażenia znajdującego się bezpośrednio po lewej stronie np:
  - `.*` oznacza dowolny ciąg znaków
  - `A[a]*` określa A, Aa, Aaa, Aaaaaaaaa, itd.
- `^` - reprezentuje początek linii
- `$` - reprezentuje koniec linii

Domyślnie `grep` pracuje w uproszczonym trybie, gdzie znaki `?`, `+`, `{`, `}`, `|`, `(` oraz `)` traktowane są dosłownie. Aby uzyskać zgodność z wyrażeniami regularnymi stosowanymi w innych językach (*extended regular expressions*),

należy stosować `grep` z przełącznikiem `-E` lub używać polecenia `egrep` (które wewnętrznie wywołuje `grep` z przełącznikiem `-E`). Rozbudowuje to dostępne znaczniki między innymi o:

- `+` - jedno lub więcej wystąpień poprzedzającego znaku
- `{n}` - n-krotne wystąpienie poprzedzającego znaku
- `{n,}` - oznacza co najmniej n-krotne wystąpienie znaku występującego bezpośrednio po lewej stronie nawiasów
- `{,m}` - oznacza co najwyżej m-krotne wystąpienie znaku występującego po lewej stronie nawiasów
- `{n,m}` - oznacza co najmniej n-krotne i co najwyżej m-krotne wystąpienie znaku występującego po lewej stronie nawiasów

Jeśli chcemy, aby znak mający znaczenie specjalne w wyrażeniach regularnych (np. `.`, `*`, `{`, `}`, `(`, `)`, `^`, `[`, `]`, `\`, `<`, `>`, `$`) został potraktowany dosłownie, musimy poprzedzić go backslashem (`\`). Ponadto, ponieważ wiele z wymienionych znaków ma w powłoce `bash` znaczenie specjalne, całe wyrażenie wpisane w konsoli należy umieścić w pojedynczym cudzysłowie (`'`), aby uniknąć ich interpretacji przez powłokę.

Przykłady

Wyświetlenie zawartości pliku wraz z numerami linii:

```
cat -n plik.txt
```

Wyświetlenie 5 pierwszych linii pliku:

```
head -n 5 plik.txt
```

Proste wyrażenie regularne - przeszukuje plik */etc/group* pod kątem słowa *video*:

```
grep video /etc/group
```

Potoki

Standardowe wyjście jednego procesu może być połączone ze standardowym wejściem innego procesu, tworząc tzw. potok pomiędzy tymi procesami.

Przetwarzanie potokowe polega na buforowaniu przez system danych produkowanych przez pierwszy proces i następnie odczytywaniu tych danych przez drugi proces. Innymi słowy proces w potoku czyta dane z wejścia, które zostało przeadresowane na wyjście procesu poprzedniego. W potoku może brać udział jednocześnie kilka procesów. Do przekierowania danych do kolejnego procesu używany jest znak `|`. Poniżej podano przykłady potoków:

Proces `ls` z przełącznikami `a` oraz `l` podaje wynik procesowi `head` z przełącznikiem `-n 2`, który w efekcie wyświetla dwie pierwsze linjki listy plików bieżącego katalogu:

```
ls -al | head -n 2
```

Proces `pwd` podaje wynik procesowi `grep`, który wyszukuje w nazwie obecnego katalogu roboczego ciąg znaków zawierających słowo `home`:

```
pwd | grep home
```

Zadanie 6: Filtry i potoki

1. Sprawdź ponownie jakie komendy uruchomiłeś podczas dzisiejszych zajęć (polecenie `history`).
2. Żeby w łatwy sposób odnaleźć wystąpienia komendy `ssh` użyj polecenia `history` a następnie przeszkuaj wystąpienia tworząc potok z komendą `grep`.

```
history | grep ssh
```

W tym przykładzie, wyjście komendy `history` jest wysłane do komendy `grep` za pomocą operatora `|` i traktowane jako jej wejście. Standardowe wyjściem całego polecenia będzie wynik komendy `grep` na wyjściu komendy `history`.

3. Pobierz plik zawierający logi https://drive.google.com/file/d/13mOZghI6jfO1p03440b424ipn5lIn330/view?u
a następnie wyświetl 10 pierwszych linii tego pliku.
4. Następnie przeszukaj jego zawartość pod kątem wystąpienia słowa `DEBUG`.
5. W kolejnym kroku stwórz potok do wyświetlenia ostatnich 5 linijek z wystąpieniem słowa `INFO` wraz z numerem linii.

Skrypty w bash

Stwórz plik `hello_world.sh` i za pomocą wybranego narzędzia tekstowego np.: `nano` wpisz do niego następującą zawartość:

```
#!/bin/bash
# declare STRING variable
STRING="Hello World"
#print variable on a screen
echo $STRING
```

Następnie nadaj mu uprawnienia do uruchomienia za pomocą komendy `chmod`:

```
chmod +x hello_world.sh
```

Uruchom skrypt:

```
./hello_world.sh
```

Co wykonał skrypt?

Zadanie 7: Skrypty w bash

1. Spróbuj zmodyfikować skrypt tak, żeby przed wyświetleniem napisu `Hello World` program wyświetlił Twoje imię i nazwisko a oraz odczekał 5 sekund (komenda `sleep`).

Autor: *Adam Bondyra, Dominik Pieczyński, Jakub Tomczyński, Mikołaj Łysakowski*

Opracowano na podstawie materiałów projektu *Otwartych Studiów Informatycznych (http://wazniak.mimuw.edu.pl/)*.